Can LLMs Learn From Mistakes? An Empirical Study on Reasoning Tasks

Anonymous ACL submission

Abstract

Towards enhancing the chain-of-though (CoT) reasoning of large language models (LLMs), much existing work has revealed the effectiveness of straightforward learning on annotated/generated CoT paths. However, there is less evidence yet that reasoning capabilities can be enhanced through a reverse learning process, i.e., learning from potential mistakes in reasoning. To investigate whether LLMs can learn from mistakes, we construct mistake-correction datasets, using GPT-4 to identify and correct the mistakes in inaccurate CoTs. With these mistake-correction datasets, we fine-tune open-source LLMs and arrive at the following conclusions. (1) LLMs can indeed learn from mistakes to enhance their CoT reasoning performances. (2) Compared to CoT data, the mistake-correction data provides additional knowledge on the explanations and reasons for the potential mistakes in CoTs, which consistently contributes to the effectiveness of learning from mistakes. (3) Evolution techniques, especially the correction-centric evolution we introduced, can further enhance the effectiveness of learning from mistakes.

1 Introduction

Mistakes are the portals of discovery.
—James Joyce

With exponential growth in data size and model scale, contemporary large language models (Brown et al., 2020; Zhang et al., 2022; Hoffmann et al., 2022; Smith et al., 2022; OpenAI, 2023b; Anil et al., 2023) have emerged the chain-of-thought (CoT) reasoning capabilities on solving complex tasks (Wei et al., 2022; Wang et al., 2022; Li et al., 2023b; Shi et al., 2023; Qin et al., 2023; Lightman et al., 2023), such as GSM8K (Cobbe et al., 2021) and MATH (Hendrycks et al., 2021). To improve the CoT reasoning of open-source LLMs such as LLaMA-2 (Touvron et al., 2023b), a common approach is to apply the **straightforward learning**, which means to directly fine-tune the models using annotated/generated CoT paths (Magister et al., 2022; Huang et al., 2022; Ho et al., 2022; Li et al., 2022; Yuan et al., 2023; Luo et al., 2023; Yu et al., 2023; Li et al., 2023a; Liang et al., 2023; Ranaldi and Freitas, 2024).

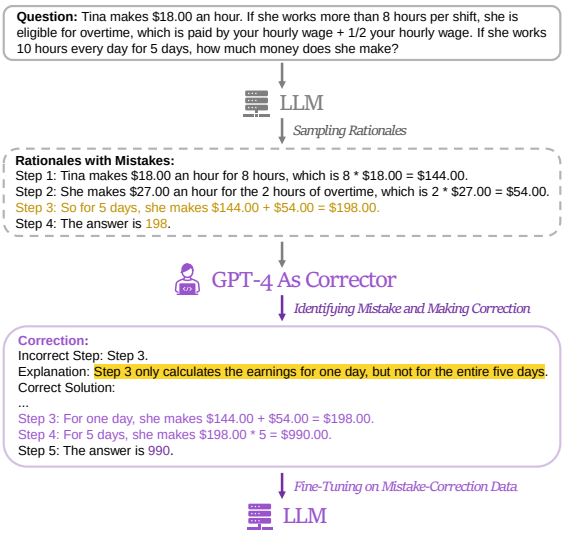


Figure 1: The overall process for investigating whether LLMs can learn from mistake. We first construct the mistake-correction dataset and then fine-tune the LLM.

Despite existing extensive research on straightforward learning processes, there remains limited exploration into whether reasoning capabilities can be enhanced through a reverse learning process, which means to **learn from what kind of mistakes could be made during reasoning**. The insight of learning from mistakes comes from the learning process of human. Consider a human student who is just beginning to learn math. Beyond learning from golden knowledge and examples in books, he also does exercises. After failing to solve a problem, he will learn what mistakes he has made and how to correct them. By learning from mistakes, his reasoning capability will be further improved. Inspired by this reverse learning process for human

students, this work explores *whether the reasoning capabilities of LLMs can also be enhanced by learning from mistakes*.

To this end, we first construct mistake-correction datasets for reasoning tasks and then use these datasets to fine-tune LLMs (illustrated in Figure 1). To construct the mistake-correction dataset, we employ multiple LLMs, including the LLaMA2 and GPT families, to collect inaccurate CoTs (i.e., with incorrect final answers). We then use GPT-4 (OpenAI, 2023b) as a "corrector" to generate corrections for these inaccurate CoTs. The corrections inform what mistakes have been made in CoTs and how to correct them. We conduct human evaluations showing that the generated corrections exhibit adequate quality for the subsequent fine-tuning stage. We then fine-tune open-source LLMs on the mistake-correction data to perform LEarning from MistAkes (LEMA), and use the augmented CoT data for straightforward learning.

Our experiments are conducted across various open-source LLMs (e.g., LLaMA2 family and specialized LLMs such as WizardMath (Luo et al., 2023) and MetaMath(Yu et al., 2023)), several reasoning tasks (including math reasoning and commonsense reasoning), and two training approaches (i.e., QLoRA (Dettmers et al., 2023) and full fine-tuning). These experiments aim to answer the following research questions.

- **RQ1: Can LLMs learn from mistakes to improve CoT reasoning? A1: Yes.** Compared to only applying straightforward learning, incorporating learning from mistakes during fine-tuning improves the reasoning performances of backbone models. Moreover, some specialized LLMs for math tasks can also be further enhanced through learning from mistakes.

- **RQ2: Why does learning from mistakes take effect? A2: It brings additional knowledge on the explanations and reasons to the potential mistakes.** Compared to the CoT data for straightforward learning, the mistake-correction data additionally provides the explanations and reasons to mistakes along with CoTs. Our ablation study reveals the importance of this additional knowledge.

- **RQ3: Can learning from mistakes benefit from evolution techniques? A3: Yes.** Despite the general evolution technique that randomly selects seed questions (Xu et al., 2023; Yu et al., 2023; Li et al., 2023a), we introduce a correction-centric evolution strategy which focuses more on moderately difficult questions. Experimental results show the further improvements from expanding the mistake-correction dataset through applying the evolution techniques.

2 Methodology

Our exploration consists of three primary stages: constructing the mistake-correction dataset, expanding the dataset with correction-centric evolution, and fine-tuning LLMs.

2.1 Mistake-Correction Data Construction

Figure 2 briefly illustrates the process of constructing the mistake-correction data. Given a question-answer example $(q_i, a_i) \in \mathcal{Q}$, a corrector model $\mathcal{M}_c$, and a reasoning model $\mathcal{M}_r$, we will generate the mistake-correction data pair $(q_i \oplus \widetilde{r}_i, c_i) \in \mathcal{C}$, where $\widetilde{r}_i$ is an inaccurate reasoning path to the question q_i, and c_i is the correction for $\widetilde{r}_i$.

Collecting inaccurate reasoning paths. We first sample multiple reasoning paths for each question q_i using the reasoning model $\mathcal{M}_r$ and retain paths not achieving the correct final answer a_i,

$$\widetilde{r}_i \sim \mathcal{M}_r(\mathcal{P}_r \oplus q_i), \quad \text{Ans}(\widetilde{r}_i) \neq a_i, \qquad (1)$$

where $\mathcal{P}_r$ is the few-shot prompt instructing the model to perform CoT reasoning, and $\text{Ans}(\cdot)$ extracts the final answer from the reasoning path.

Generating corrections for mistakes. For question q_i and the inaccurate reasoning path $\widetilde{r}_i$, we employ the corrector model $\mathcal{M}_c$ to generate a correction and check the final answer in the correction,

$$c_i \sim \mathcal{M}_c(\mathcal{P}_c \oplus q_i \oplus \widetilde{r}_i), \quad \text{Ans}(c_i) = a_i, \qquad (2)$$

where $\mathcal{P}_c$ contains 4 annotated mistake-correction examples to guide the corrector model what kind of information should be contained in the generated corrections. Figure 3 briefly illustrates $\mathcal{P}_c$. Specifically, the annotated corrections comprises two pieces of information:

- **Explanation and reason to mistake**: identify which step is incorrect and explain what kind of mistake has been made in this step.

- **Correct solution**: revise the original reasoning path to achieve the correct answer.

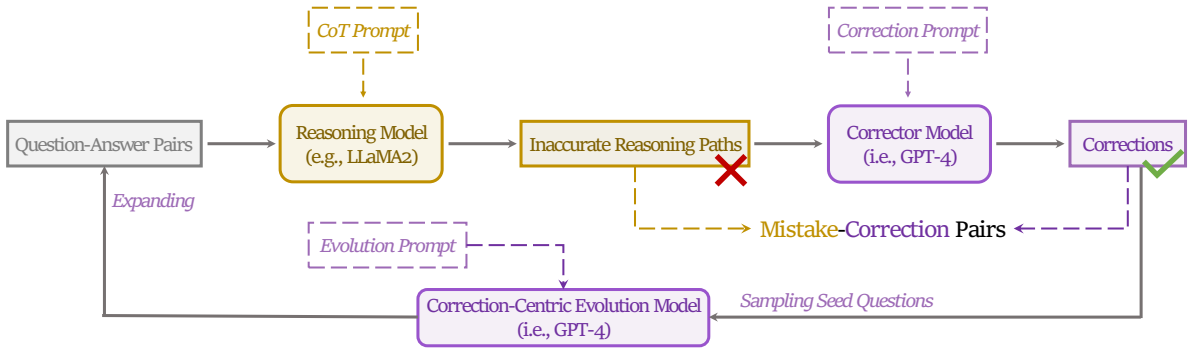


Figure 2: Process of generating and expanding the mistake-correction dataset.

Human evaluation for generated corrections. Before generating data on a large scale, we first manually assess the quality of the generated corrections. We take LLaMA-2-70B as $\mathcal{M}_r$, utilize GPT-4 as $\mathcal{M}_c$, and generate 50 mistake-correction data pairs based on the GSM8K training set. We classify the corrections into three quality levels.

- **Excellent**: the corrector successfully identifies the incorrect step in $\widetilde{r}_i$, provides a reasonable explanation, and the corrected reasoning path exhibits high continuity with the pre-steps in the original reasoning path[1].
- **Good**: the corrector successfully identifies the incorrect step in $\widetilde{r}_i$, provides a reasonable explanation, while the corrected reasoning path has minor issues in continuity.
- **Poor**: the corrector fails to identify the incorrect step in $\widetilde{r}_i$ or provides unreasonable explanations.

Appendix B.1 lists several examples under each quality level. Our evaluation finds that 35 out of 50 generated corrections are of excellent quality, 11 are good, and 4 are poor. Based on this human evaluation, we suppose the overall quality of corrections generated with GPT-4 is sufficient for the further fine-tuning stage. We generate corrections on a large scale and take all corrections that have correct final answers for fine-tuning LLMs. We provide further analysis on the choice and behavior of corrector model in Section D.8.

2.2 Correction-Centric Evolution

After building up the data generation pipeline, we explore how to scale up our correction data. We consider that expanding the question-answer set $\mathcal{Q}$ is a promising direction, as it primarily determines the correction data diversity.

Inspired by the recent success of evolution techniques on CoT augmentation (Xu et al., 2023; Yu et al., 2023; Li et al., 2023a), we explore how to effectively apply the evolution method to expand our correction data. The "evolution" means to generate a set of new question-answer pairs from the given *seed questions* by prompting powerful LLMs.

The general evolution method for CoT augmentation randomly selects seed questions to evolve. However, this strategy does not well suit the nature of our correction data, as too simple or too challenging questions are less valuable for evolving and collecting correction information.

- For too simple questions, the reasoning models such as LLaMA can already solve them. Evolving these questions may not be effective for collecting mistakes.
- For too challenging questions, the most powerful LLMs still cannot handle them. Evolving these questions may lead to much inaccurate information in corrections.

Therefore, we apply a **correction-centric evolution** strategy which more focuses on moderately difficult questions: *we only sample seed questions that occur in our correction data* $\mathcal{C}$, rather than randomly sampling from the entire set $\mathcal{Q}$,

$$\hat{q}_i \sim \mathcal{M}_e(\mathcal{P}_e \oplus q_i), \quad q_i \in \mathcal{C}, \tag{3}$$

where q_i is the seed question, and $\mathcal{M}_e$ and $\mathcal{P}_e$ are the LLM and prompt for evolving questions, respectively. Appendix B.3 illustrates our $\mathcal{P}_e$.

The underlying principle of this strategy is straightforward. If one question frequently appears in correction data, it means that this question is not well solved by many reasoning models, but its

[1]The high continuity means that the corrected reasoning steps follow the pre-steps generated before the identified mistake step.

Prompt For Generating Corrections

For the following math word problems, the original solutions may contain errors. Please identify the incorrect step in each solution, explain why it is incorrect, and provide the correct solution starting from that step.

Question: James creates a media empire. He creates a movie for $2000. Each DVD cost $6 to make. He sells it for 2.5 times that much. He sells 500 movies a day for 5 days a week. How much profit does he make in 20 weeks?

Original Solution: Step 1, 500 movies a day, 5 days a week, for 20 weeks, he sells 500 * 5 * 20 = 50000 movies. Step 2, Each movie he sells for $6 * 2.5 = $15. Step 3, Thus, he makes a profit of $15 - $6 = $9 per movie. Step 4, Therefore, he makes a profit of $9 * 50000 = $450000. Step 5, The answer is 450000.

Incorrect Step: Step 4.

Explanation: The error in Step 4 is that it does not take into account the initial cost of creating the movie ($2000). The correct solution should subtract this amount from the total profit calculated in Step 4.

Correct Solution: … Step 4, Therefore, he makes a profit of $9 * 50000 - $2000 = $448000. Step 5, The answer is 448000.
… (Another 3 annotated examples)

Question: q_i, **Original Solution:** $\tilde{r}_i$, **Incorrect Step:** {Completion}

Figure 3: A brief illustration of our prompt for generating corrections. For the incorrect step in the original solution, we prompt GPT-4 to find out the reason of mistake and generate the corrected step.

inaccurate reasoning paths can be well handled by the corrector model.

2.3 Fine-Tuning LLMs

After generating the correction data, we fine-tune LLMs to examine whether these correction data can facilitate CoT reasoning. We compare the results under two settings:

- **Only straightforward learning**. We fine-tune the model on CoT data alone. In addition to the annotated data in each task, we additionally take CoT data augmentation following existing methods (Yuan et al., 2023; Li et al., 2023a; Yu et al., 2023). We generate more reasoning paths for each question in the training sets with GPT-4 and filter out paths with wrong final answers. We apply this CoT data augmentation to set up strong baselines for straightforward learning.

- **Incorporating learning from mistakes**. We fine-tune LLMs on both CoT data and the constructed mistake-correction dataset. This setting is referred to as LEMA.

Appendix B.2 shows the input-output formats of CoT data and mistake-correction data used for fine-tuning and evaluation.

3 Experimental Setup

3.1 Tasks

We undertake experiments on three challenging reasoning tasks[2], including two mathematical reasoning tasks, GSM8K and MATH, and one commonsense reasoning task CSQA. For these tasks, we generate correction data based on their training sets. Despite these tasks, we also take two additional tasks (SVAMP and ASDiv) for out-of-distribution evaluation (detailed in Section D.3).

GSM8K (Cobbe et al., 2021) contains high quality linguistically diverse grade school math word problems. It has 7,473 training examples with CoT and 1,319 test cases.

MATH (Hendrycks et al., 2021) examines math reasoning on solving challenging competition mathematics problems. It contains 7,500 training CoT data and 5,000 test cases.

CSQA (Talmor et al., 2019) is a question answering dataset for commonsense reasoning. It has 9,741 examples in the training set and 1,221 examples in the dev set. As it does not contain any CoT annotation, we first annotate 4 CoT examples (detailed in Appendix C.3), then take its training set to augment CoT data and generate correction data.

3.2 Data Construction

CoT Data. For GSM8K, the CoT data contains all training examples of GSM8K and 24,948 augmented reasoning paths. We first generate 30,000 reasoning paths with GPT-4 and filter out 5,052 paths with wrong final answers or unexpected format[3]. For MATH, the CoT data contains all training examples and 12,509 augmented reasoning paths. We sample 30,000 reasoning paths with GPT-4 and filter out 17,491 paths. For CSQA, we generate 15,000 reasoning paths with GPT-4 and then filter out 4,464 paths.

Mistake-Correction Data. We utilize multiple LLMs to collect inaccurate reasoning paths, in-

[2]Appendix C.1 contains basic statics about these tasks.

[3]The unexpected format means that the final answer is failed to be extracted from the path with the regular expression.

Table 1: Our main experimental results (%) on three reasoning tasks. Appendix D.3 contains the results on another two out-of-distribution tasks, and Appendix D.1 and D.2 illustrate the performance variances during training.

Model	Training	Method	GSM8K		MATH		CSQA		Average	
			Acc	Δ	Acc	Δ	Acc	Δ	Acc	Δ
LLaMA-2-70B (Touvron et al., 2023b)	QLoRA	Straightforward Learning	81.4	-	23.6	-	84.2	-	63.1	-
		+ Learning From Mistakes	83.5	**+2.1**	25.0	**+1.4**	85.3	+1.1	64.6	**+1.5**
LLaMA-65B (Touvron et al., 2023a)	QLoRA	Straightforward Learning	76.2	-	19.7	-	83.1	-	59.7	-
		+ Learning From Mistakes	77.9	+1.7	20.8	**+1.1**	84.0	+0.9	60.9	+1.2
CodeLLaMA-34B (Rozière et al., 2023)	QLoRA	Straightforward Learning	68.8	-	19.1	-	78.1	-	55.3	-
		+ Learning From Mistakes	71.7	**+2.9**	20.4	**+1.3**	80.8	**+2.7**	57.6	**+2.3**
LLaMA-2-13B (Touvron et al., 2023b)	Full Fine-Tuning	Straightforward Learning	63.6	-	14.0	-	80.1	-	52.6	-
		+ Learning From Mistakes	67.0	**+3.4**	16.5	**+2.5**	82.1	**+2.0**	55.2	**+2.6**
	QLoRA	Straightforward Learning	62.9	-	12.2	-	80.4	-	51.8	-
		+ Learning From Mistakes	65.7	**+2.8**	12.6	+0.4	81.9	+1.5	53.4	**+1.6**
LLaMA-2-7B (Touvron et al., 2023b)	Full Fine-Tuning	Straightforward Learning	55.0	-	10.1	-	76.9	-	47.3	-
		+ Learning From Mistakes	57.1	**+2.1**	11.6	**+1.5**	79.0	**+2.1**	49.2	**+1.9**
	QLoRA	Straightforward Learning	52.6	-	8.7	-	76.9	-	46.1	-
		+ Learning From Mistakes	54.1	+1.5	9.4	+0.7	78.8	+1.9	47.4	+1.3

Table 2: Performances of LEMA with specialized LLMs on GSM8K (with QLoRA for training).

Model	Acc (%)
WizardMath-70B (Luo et al., 2023)	81.6
WizardMath-70B + LEMA	**84.2 (+2.6)**
MetaMath-70B (Yu et al., 2023)	82.3
MetaMath-70B + LEMA	**85.4 (+3.1)**

cluding LLaMA-2 (Touvron et al., 2023b), WizardLM (Xu et al., 2023), WizardMath (Luo et al., 2023), Text-Davinci-003 (OpenAI, 2023c), GPT-3.5-Turbo (OpenAI, 2023a) and GPT-4 (OpenAI, 2023b). We take GPT-4 as the corrector model. Finally, we collect 12,523, 6,306, 7,241 mistake-correction pairs based on the training sets of GSM8K, MATH and CSQA, respectively.

Correction-Centric Evolution. We take 10K bootstrap samples from the questions in our correction data. We utilize GPT-4 to evolve the questions. To generate "ground-truth" answers for the evolved questions, we utilize GPT-4 to sample three answers for each question and conduct a majority voting. The question that leads to three different answers will be filtered. Note that the evolved data will only be used in Section 4.3.

3.3 Fine-Tuning and Evaluation

We fine-tune multiple open-source LLMs in the LLaMA (Touvron et al., 2023a), LLaMA-2 (Touvron et al., 2023b), CodeLLaMA (Rozière et al., 2023), WizardMath (Luo et al., 2023) and MetaMath (Yu et al., 2023) families. We consider SFT rather than DPO for fine-tuning, as DPO only uses the correct and mistake CoTs during training while the explanations and reasons of mistakes are not used.

The fine-tuning approaches cover both QLoRA and full fine-tuning. Considering the high training cost, the full fine-tuning is mainly applied on relative small models (such as 7B and 13B models).

QLoRA fine-tuning. We use QLoRA[4] (Hu et al., 2022; Dettmers et al., 2023) to conduct parameter-efficient fine-tuning (PEFT). We set low-rank dimension as 64 and dropout rate as 0.05. We set learning rate as 1e-4 for LLMs larger than (or equal to) 34B and 2e-4 for LLMs smaller than 34B. We set batch size as 96, train for 2K steps, and save checkpoints for every 100 training steps. We evaluate the performance of all saved checkpoints and report the accuracy of the best checkpoint. To clarify the influence from random disturbances during training, we provide the performances of the best three checkpoints in Appendix D.1 and the performance curves during the whole training processes in Appendix D.2.

Full fine-tuning. We set learning rate as 1e-5 for LLMs larger than (or equal to) 34B and 2e-5 for LLMs smaller than 34B, and set batch size as 128. To avoid severe over-fitting problem, we apply

[4] https://github.com/artidoro/qlora.

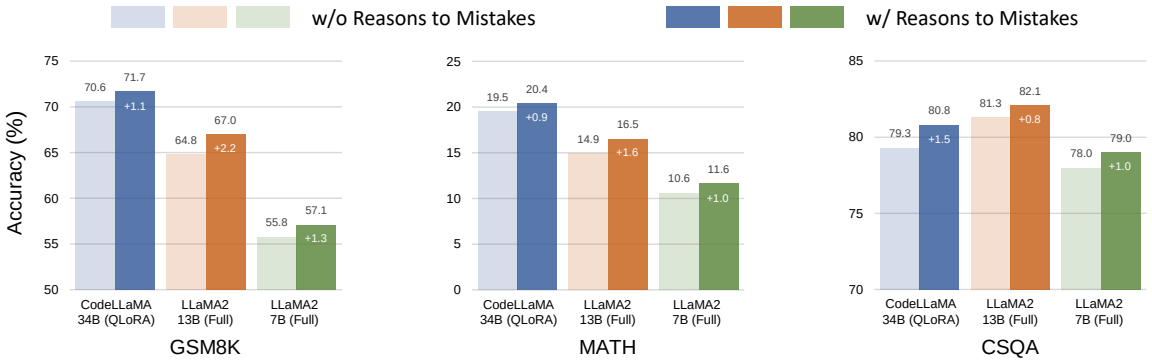


Figure 4: Performances of LEMA with (w/) and without (w/o) providing the reasons to the mistakes. Across various models and tasks, the reasons to the mistakes consistently contribute to the performances.

a cosine learning rate scheduler and only take 3-epoch training. The final checkpoint will be saved and evaluated.

For evaluation, we take vLLM library[5] (Kwon et al., 2023) for efficient inference. We set temperature as 0 (i.e., greedy decoding) and max sample length as 2,048. We do not add demonstration examples into the prompt for both fine-tuning and evaluation by default. All evaluations are conducted under the same CoT instruction. For models trained with LEMA, we do not generate corrections during evaluations. All our experiments can be conducted on 4 x A100 GPU stations.

4 Results and Analysis

We mainly focus on three main research questions in this section. More results and analysis are contained in Appendix D.

4.1 RQ1: Can LLMs Learn From Mistakes?

The following experimental results show that incorporating learning from mistakes can improve CoT reasoning. These results demonstrate that LLMs can learn from mistakes to improve their reasoning performances.

Incorporating learning from mistakes effectively improves CoT reasoning for various base models. Table 1 shows the main experimental results for five base models and three reasoning tasks. Compared to only applying the straightforward learning, incorporating learning from mistakes during fine-tuning brings improvements across all models and tasks. Such improvements demonstrate that these base model can indeed benefit from learning from mistakes. Despite these in-task performances, Appendix D.3 shows the improvements on unseen tasks.

Specialized LLMs can be further enhanced through learning from mistakes. To adapt base models into the math domain, there have been several specialized LLMs such as WizardMath (Luo et al., 2023) and MetaMath (Yu et al., 2023). We also apply the mistake-correction dataset to fine-tune these specialized LLMs. As these models have been already enhanced through straightforward learning, here we directly compare with the results reported in the original papers for these specialized models. Table 2 shows that by incorporating learning from mistakes, the performances of these specialized LLMs can be further improved. Appendix D.6 contains more experimental results on specialized LLMs.

SFT vs. DPO. Despite implementing learning from mistakes via SFT, we also try DPO using the mistake-correction dataset. Our preliminary exploration in Appendix D.4 shows that SFT training outperforms the DPO and its variant. We suppose it is because the DPO training cannot fully utilize the mistake-correction data, specifically, the explanations and reasons of mistakes are discarded. In the following section, we will further demonstrate the imporatance of this part of additional knowledge.

4.2 RQ2: Why Does Learning From Mistakes Take Effect?

Compared to straightforward learning which only learns from CoTs, learning from mistakes provides both the corrected CoTs and **the explanations and reasons to mistakes, thus providing additional knowledge for the model**. The following experiments demonstrate that such kind of additional knowledge contributes to the reasoning per-

[5] https://github.com/vllm-project/vllm.

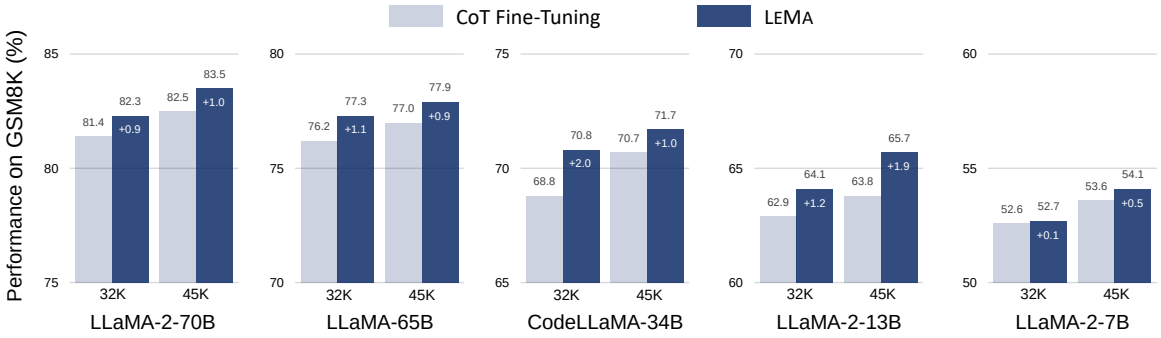


Figure 5: Performances of LEMA and only straightforward learning with controlled data sizes (32K and 45K) on GSM8K (with QLoRA for training). These gains further demonstrate the effectiveness of the additional knowledge provided by LEMA (i.e., the explanations and reasons to the mistakes).

formances of LLMs.

Ablating the explanations and reasons of mistakes affect the effectiveness of learning from mistakes. For the two part of information in mistake-correction dataset, here we discard the explanations and reasons to mistakes and keep the corrected solutions. As shown in Figure 4, across different models and tasks, the performances consistently decrease when the explanations and reasons to mistakes are not used during fine-tuning. Such decrements are in line with the performances of DPO training (see Appendix D.4) which just use correct and mistake CoTs. These results demonstrate that the additional knowledge provided by the explanations and reasons to mistakes indeed contribute to the reasoning performances of LLMs.

Mistake-correction data has non-homogeneous effectiveness with CoT data. If the effectiveness of the two data sources are completely homogeneous, the gains in Table 1 will be diminished if the data sizes for two learning processes are controlled as the same. To further validate the effectiveness of mistake-correction data, we conduct two ablation studies with controlled data sizes. In default settings, we have about 32K examples for CoT-alone fine-tuning and 45K examples for LEMA. Here are another two controlled settings:

- LEMA-32K. We keep the 13K correction data and randomly remove 13K CoT data.
- CoT-45K. To expand CoT data, we extract the corrected CoT from each correction example.

Figure 5 shows that LEMA can still bring gains under the same data size. Note that under the same data size, the two learning processes contain the same amount of CoTs at the target side, while learning from mistakes additionally provides the explanations and reasons to mistakes. These results further demonstrate that the additional knowledge from learning from mistakes improves the reasoning performances of LLMs. Despite controlling the training data sizes to be the same, Appendix D.5 provides further investigation about the training-token efficiency of learning from mistakes.

4.3 RQ3: Can Learning From Mistakes Benefit From Evolution Techniques?

We apply two evolution techniques to expand the mistake-correction dataset. The following experiments demonstrate that learning from mistakes can be further improved with evolution techniques.

Evolution techniques, especially the correction-centric evolution, can further improve the performance of leaning from mistakes. Figure 6 shows the performance of learning from mistakes with incorporating evolution techniques[6]. There are two primary conclusions. First, learning from mistakes can effectively benefit from evolution techniques. It indicates that the performance of LEMA can be further improved by incorporating existing data augmentation techniques. Second, the correction-centric evolution outperforms the general evolution. It demonstrates that moderately difficult questions are more suitable for expanding the correction data.

Learning from mistakes has a better scaling trend under full fine-tuning. To explore the scaling trend of learning from mistakes, we apply the correction-centric evolution on another 10K sampled seed questions (detailed in Appendix C.5). Figure 7 shows the performance trends of LEMA

[6]Appendix C.4 contains the detailed experimental settings.

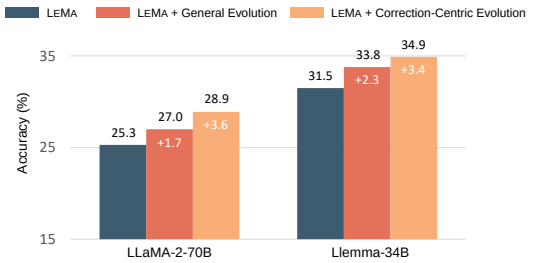


Figure 6: Performance of LEMA on MATH with general and correction-centric evolution (with full fine-tuning).

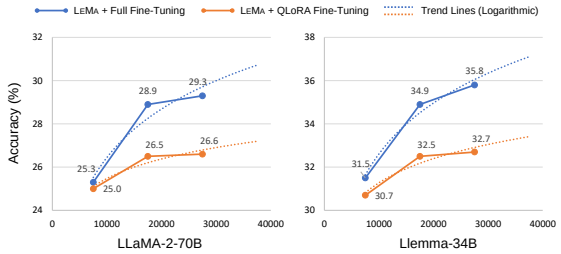


Figure 7: The performance trends of LEMA with QLoRA and full fine-tuning (logarithmically fitted). X-axis is the number of sampled questions.

as the question set expands. It shows that with expanding the question set, the performances with full fine-tuning improve significantly, while the performances with QLoRA increase slightly. It indicates that the parameter-efficient fine-tuning can only "digest" a limited scale of correction data. Appendix D.7 provides further analysis.

5 Related Work

LLMs with CoT reasoning. Wei et al. (2022) uncovered the emergence of CoT reasoning capability for extremely large language models, and this reasoning capability was then examined in various reasoning-related domains including logical reasoning (Creswell et al., 2022; Pan et al., 2023; Lei et al., 2023), commonsense reasoning (Talmor et al., 2019; Geva et al., 2021; Ahn et al., 2022), and math reasoning (Miao et al., 2020; Koncel-Kedziorski et al., 2016; Patel et al., 2021; Cobbe et al., 2021; Hendrycks et al., 2021). The impressive performance of LLMs in these domains has spurred the research community to further investigate methods for effectively harnessing and enhancing CoT reasoning for LLMs (Wang et al., 2022; Zhou et al., 2022; Creswell and Shanahan, 2022; Li et al., 2023b; Lightman et al., 2023).

Enhancing CoT reasoning for solving mathematical problems. There has been much work dedicated to enhancing the performance of LLMs in solving mathematical problems from various perspectives. Some studies explored the voting or verification methods based on sampling multiple reasoning paths (Wang et al., 2022; Li et al., 2023b; Lightman et al., 2023). Some methods considered to generate executable programs to obtain the final answer or to integrate plug-in tools that facilitate the execution of external APIs during intermediate steps (Jie and Lu, 2023; Wang et al., 2023a; Yue et al., 2023; Azerbayev et al., 2023; Gou et al., 2023). Some work collected math-related corpus such as arXiv papers for pre-training better base models for math (Azerbayev et al., 2023; Wang et al., 2023d). Some work focused on augmenting existing datasets, which expanded training sets or provided external annotations (Magister et al., 2022; Huang et al., 2022; Ho et al., 2022; Li et al., 2022; Luo et al., 2023; Yu et al., 2023; Li et al., 2023a; Liang et al., 2023; Liu et al., 2023a,b; Wang et al., 2023e). From the perspective of the techniques used, this work follows the data augmentation approach.

Using mistake data to improve LLMs. Some recent work has explored how to leverage the mistake data to improve the performances of LLMs, , instead of merely relying on the correct data. There are primarily three existing ways of using mistake data: 1) prompt and agent engineering such as Reflexion (Shinn et al., 2024) and SALAM (Wang and Li, 2023), which exploits the incorrect attempts in historical data to improve the performance of a frozen LLM; 2) training verifiers (Cobbe et al., 2021; Lightman et al., 2023), which fine-tunes a small model to re-rank the candidate answers from the LLMs; 3) preference optimization (Rafailov et al., 2023; Meng et al., 2024; Pal et al., 2024; Yuan et al., 2024), which modifies the training objective with incorporating the model preferences on mistake data. To the best of our knowledge, we are the first to explore whether the mistake reasoning data can be directly utilized through a standard fine-tuning approach.

6 Conclusion

This work provides an empirical study on exploring whether LLMs can learn from mistakes to improve their CoT reasoning performances. Our experiments on mistake-correction data reveal that LLMs can indeed learn from mistakes, especially benefit from the explanations and reasons to mistakes.

Limitations

Relying on GPT-4 for data construction. Our data generation process heavily relies on calling GPT-4 API. Moreover, this reliance can not be replaced with some cheaper APIs such as GPT-3.5-Turbo (analyzed in Appendix D.8). We consider the self-correction/self-reflection methods as future directions for learning from mistakes.

Limitation on data scale. The data scale we explored is <100K. This is mainly due to the low success rate for correcting mistakes from challenging questions (analyzed in Appendix D.8) and also the high cost for GPT-4 API calling. Our experiments with evolution techniques in Section 4.3 implies the potential of LEMA on a larger data scale.

Potential performance degradation due to the parameter-efficient tuning. Our experiments in Table 1 and Figure 7 indicate that the QLoRA fine-tuning might limit the effectiveness of learning from mistakes. Much existing work also revealed that parameter-efficient tuning might affect the final performance, especially for smaller models (Lester et al., 2021; An et al., 2022; Sun et al., 2023; Su et al., 2023; Artur Niederfahrenhorst and Ahmad, 2023). This might limit the performances shown in Table 1 and Table 2.

Ethics Statement

Due to the using of pre-trained language models, this work could be exposed to some potential risks of ethical issues on general deep learning models (such as social bias and privacy breaches). We hope that the idea of learning from mistakes would facilitate the development of responsible AI models, for instance, on training LLMs to recognize and modify risky generated contents.

References

Michael Ahn, Anthony Brohan, Noah Brown, Yevgen Chebotar, Omar Cortes, Byron David, Chelsea Finn, Chuyuan Fu, Keerthana Gopalakrishnan, Karol Hausman, Alex Herzog, Daniel Ho, Jasmine Hsu, Julian Ibarz, Brian Ichter, Alex Irpan, Eric Jang, Rosario Jauregui Ruano, Kyle Jeffrey, Sally Jesmonth, Nikhil J Joshi, Ryan Julian, Dmitry Kalashnikov, Yuheng Kuang, Kuang-Huei Lee, Sergey Levine, Yao Lu, Linda Luu, Carolina Parada, Peter Pastor, Jornell Quiambao, Kanishka Rao, Jarek Rettinghouse, Diego Reyes, Pierre Sermanet, Nicolas Sievers, Clayton Tan, Alexander Toshev, Vincent Vanhoucke, Fei Xia, Ted Xiao, Peng Xu, Sichun Xu, Mengyuan Yan, and Andy Zeng. 2022. Do as i can, not as i say: Grounding language in robotic affordances.

Alibaba. 2023. Alibaba open sources qwen, a 7b parameter ai model.

Shengnan An, Yifei Li, Zeqi Lin, Qian Liu, Bei Chen, Qiang Fu, Weizhu Chen, Nanning Zheng, and Jian-Guang Lou. 2022. Input-tuning: Adapting unfamiliar inputs to frozen pretrained models.

Rohan Anil, Andrew M. Dai, Orhan Firat, Melvin Johnson, Dmitry Lepikhin, Alexandre Passos, Siamak Shakeri, Emanuel Taropa, Paige Bailey, Zhifeng Chen, Eric Chu, Jonathan H. Clark, Laurent El Shafey, Yanping Huang, Kathy Meier-Hellstern, Gaurav Mishra, Erica Moreira, Mark Omernick, Kevin Robinson, Sebastian Ruder, Yi Tay, Kefan Xiao, Yuanzhong Xu, Yujing Zhang, Gustavo Hernandez Abrego, Junwhan Ahn, Jacob Austin, Paul Barham, Jan Botha, James Bradbury, Siddhartha Brahma, Kevin Brooks, Michele Catasta, Yong Cheng, Colin Cherry, Christopher A. Choquette-Choo, Aakanksha Chowdhery, Clément Crepy, Shachi Dave, Mostafa Dehghani, Sunipa Dev, Jacob Devlin, Mark Díaz, Nan Du, Ethan Dyer, Vlad Feinberg, Fangxiaoyu Feng, Vlad Fienber, Markus Freitag, Xavier Garcia, Sebastian Gehrmann, Lucas Gonzalez, Guy Gur-Ari, Steven Hand, Hadi Hashemi, Le Hou, Joshua Howland, Andrea Hu, Jeffrey Hui, Jeremy Hurwitz, Michael Isard, Abe Ittycheriah, Matthew Jagielski, Wenhao Jia, Kathleen Kenealy, Maxim Krikun, Sneha Kudugunta, Chang Lan, Katherine Lee, Benjamin Lee, Eric Li, Music Li, Wei Li, YaGuang Li, Jian Li, Hyeontaek Lim, Hanzhao Lin, Zhongtao Liu, Frederick Liu, Marcello Maggioni, Aroma Mahendru, Joshua Maynez, Vedant Misra, Maysam Moussalem, Zachary Nado, John Nham, Eric Ni, Andrew Nystrom, Alicia Parrish, Marie Pellat, Martin Polacek, Alex Polozov, Reiner Pope, Siyuan Qiao, Emily Reif, Bryan Richter, Parker Riley, Alex Castro Ros, Aurko Roy, Brennan Saeta, Rajkumar Samuel, Renee Shelby, Ambrose Slone, Daniel Smilkov, David R. So, Daniel Sohn, Simon Tokumine, Dasha Valter, Vijay Vasudevan, Kiran Vodrahalli, Xuezhi Wang, Pidong

Wang, Zirui Wang, Tao Wang, John Wieting, Yuhuai Wu, Kelvin Xu, Yunhan Xu, Linting Xue, Pengcheng Yin, Jiahui Yu, Qiao Zhang, Steven Zheng, Ce Zheng, Weikang Zhou, Denny Zhou, Slav Petrov, and Yonghui Wu. 2023. Palm 2 technical report.

Anthropic. 2023. Model card and evaluations for claude models.

Kourosh Hakhamaneshi Artur Niederfahrenhorst and Rehaan Ahmad. 2023. Fine-tuning llms: Lora or full-parameter? an in-depth analysis with llama 2.

Zhangir Azerbayev, Hailey Schoelkopf, Keiran Paster, Marco Dos Santos, Stephen McAleer, Albert Q. Jiang, Jia Deng, Stella Biderman, and Sean Welleck. 2023. Llemma: An open language model for mathematics.

Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, et al. 2020. Language models are few-shot learners. Advances in neural information processing systems, 33:1877–1901.

Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Lukasz Kaiser, Matthias Plappert, Jerry Tworek, Jacob Hilton, Reiichiro Nakano, et al. 2021. Training verifiers to solve math word problems. arXiv preprint arXiv:2110.14168.

Antonia Creswell and Murray Shanahan. 2022. Faithful reasoning using large language models. arXiv preprint arXiv:2208.14271.

Antonia Creswell, Murray Shanahan, and Irina Higgins. 2022. Selection-inference: Exploiting large language models for interpretable logical reasoning. In The Eleventh International Conference on Learning Representations.

Tim Dettmers, Artidoro Pagnoni, Ari Holtzman, and Luke Zettlemoyer. 2023. Qlora: Efficient finetuning of quantized llms. arXiv preprint arXiv:2305.14314.

Mor Geva, Daniel Khashabi, Elad Segal, Tushar Khot, Dan Roth, and Jonathan Berant. 2021. Did Aristotle Use a Laptop? A Question Answering Benchmark with Implicit Reasoning Strategies. Transactions of the Association for Computational Linguistics, 9:346–361.

Zhibin Gou, Zhihong Shao, Yeyun Gong, Yelong Shen, Yujiu Yang, Minlie Huang, Nan Duan, and Weizhu Chen. 2023. Tora: A tool-integrated reasoning agent for mathematical problem solving.

Dan Hendrycks, Collin Burns, Saurav Kadavath, Akul Arora, Steven Basart, Eric Tang, Dawn Song, and Jacob Steinhardt. 2021. Measuring mathematical problem solving with the math dataset. In Thirty-fifth Conference on Neural Information Processing Systems Datasets and Benchmarks Track (Round 2).

Namgyu Ho, Laura Schmid, and Se-Young Yun. 2022. Large language models are reasoning teachers. arXiv preprint arXiv:2212.10071.

Jordan Hoffmann, Sebastian Borgeaud, Arthur Mensch, Elena Buchatskaya, Trevor Cai, Eliza Rutherford, Diego de Las Casas, Lisa Anne Hendricks, Johannes Welbl, Aidan Clark, et al. 2022. Training compute-optimal large language models. arXiv preprint arXiv:2203.15556.

Edward J Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. 2022. LoRA: Low-rank adaptation of large language models. In International Conference on Learning Representations.

Jiaxin Huang, Shixiang Shane Gu, Le Hou, Yuexin Wu, Xuezhi Wang, Hongkun Yu, and Jiawei Han. 2022. Large language models can self-improve. arXiv preprint arXiv:2210.11610.

Jie Huang, Xinyun Chen, Swaroop Mishra, Huaixiu Steven Zheng, Adams Wei Yu, Xinying Song, and Denny Zhou. 2023. Large language models cannot self-correct reasoning yet.

Zhanming Jie and Wei Lu. 2023. Leveraging training data in few-shot prompting for numerical reasoning. arXiv preprint arXiv:2305.18170.

Rik Koncel-Kedziorski, Subhro Roy, Aida Amini, Nate Kushman, and Hannaneh Hajishirzi. 2016. MAWPS: A math word problem repository. In Proceedings of the 2016 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, pages 1152–1157, San Diego, California. Association for Computational Linguistics.

Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. 2023. Efficient memory management for large language model serving with pagedattention. In Proceedings of the ACM SIGOPS 29th Symposium on Operating Systems Principles.

Harrison Lee, Samrat Phatale, Hassan Mansoor, Thomas Mesnard, Johan Ferret, Kellie Lu, Colton Bishop, Ethan Hall, Victor Carbune, Abhinav Rastogi, and Sushant Prakash. 2023. Rlaif: Scaling reinforcement learning from human feedback with ai feedback.

Bin Lei, Chunhua Liao, Caiwen Ding, et al. 2023. Boosting logical reasoning in large language models through a new framework: The graph of thought. arXiv preprint arXiv:2308.08614.

Brian Lester, Rami Al-Rfou, and Noah Constant. 2021. The power of scale for parameter-efficient prompt tuning. In Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing, pages 3045–3059.

Chengpeng Li, Zheng Yuan, Guanting Dong, Keming Lu, Jiancan Wu, Chuanqi Tan, Xiang Wang, and Chang Zhou. 2023a. Query and response augmentation cannot help out-of-domain math reasoning generalization. arXiv preprint arXiv:2310.05506.

Shiyang Li, Jianshu Chen, Yelong Shen, Zhiyu Chen, Xinlu Zhang, Zekun Li, Hong Wang, Jing Qian, Baolin Peng, Yi Mao, et al. 2022. Explanations from large language models make small reasoners better. arXiv preprint arXiv:2210.06726.

Yifei Li, Zeqi Lin, Shizhuo Zhang, Qiang Fu, Bei Chen, Jian-Guang Lou, and Weizhu Chen. 2023b. Making language models better reasoners with step-aware verifier. In Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), pages 5315–5333.

Zhenwen Liang, Wenhao Yu, Tanmay Rajpurohit, Peter Clark, Xiangliang Zhang, and Ashwin Kaylan. 2023. Let gpt be a math tutor: Teaching math word problem solvers with customized exercise generation. arXiv preprint arXiv:2305.14386.

Hunter Lightman, Vineet Kosaraju, Yura Burda, Harri Edwards, Bowen Baker, Teddy Lee, Jan Leike, John Schulman, Ilya Sutskever, and Karl Cobbe. 2023. Let's verify step by step.

Bingbin Liu, Sebastien Bubeck, Ronen Eldan, Janardhan Kulkarni, Yuanzhi Li, Anh Nguyen, Rachel Ward, and Yi Zhang. 2023a. Tinygsm: achieving >80

Yixin Liu, Avi Singh, C. Daniel Freeman, John D. Co-Reyes, and Peter J. Liu. 2023b. Improving large language model fine-tuning for solving math problems.

Ximing Lu, Sean Welleck, Jack Hessel, Liwei Jiang, Lianhui Qin, Peter West, Prithviraj Ammanabrolu, and Yejin Choi. 2022. Quark: Controllable text generation with reinforced unlearning. In Advances in Neural Information Processing Systems.

Haipeng Luo, Qingfeng Sun, Can Xu, Pu Zhao, Jianguang Lou, Chongyang Tao, Xiubo Geng, Qingwei Lin, Shifeng Chen, and Dongmei Zhang. 2023. Wizardmath: Empowering mathematical reasoning for large language models via reinforced evol-instruct. arXiv preprint arXiv:2308.09583.

Lucie Charlotte Magister, Jonathan Mallinson, Jakub Adamek, Eric Malmi, and Aliaksei Severyn. 2022. Teaching small language models to reason. arXiv preprint arXiv:2212.08410.

Yu Meng, Mengzhou Xia, and Danqi Chen. 2024. Simpo: Simple preference optimization with a reference-free reward. arXiv preprint arXiv:2405.14734.

Shen-yun Miao, Chao-Chun Liang, and Keh-Yih Su. 2020. A diverse corpus for evaluating and developing English math word problem solvers. In Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics, pages 975–984, Online. Association for Computational Linguistics.

OpenAI. 2023a. Gpt-3.5 turbo fine-tuning and api updates.

OpenAI. 2023b. Gpt-4 technical report.

OpenAI. 2023c. Openai documentation: Models.

Arka Pal, Deep Karkhanis, Samuel Dooley, Manley Roberts, Siddartha Naidu, and Colin White. 2024. Smaug: Fixing failure modes of preference optimisation with dpo-positive.

Liangming Pan, Alon Albalak, Xinyi Wang, and William Yang Wang. 2023. Logic-lm: Empowering large language models with symbolic solvers for faithful logical reasoning. arXiv preprint arXiv:2305.12295.

Arkil Patel, Satwik Bhattamishra, and Navin Goyal. 2021. Are NLP models really able to solve simple math word problems? In Proceedings of the 2021 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, pages 2080–2094, Online. Association for Computational Linguistics.

Chengwei Qin, Aston Zhang, Zhuosheng Zhang, Jiaao Chen, Michihiro Yasunaga, and Diyi Yang. 2023. Is chatgpt a general-purpose natural language processing task solver? arXiv preprint arXiv:2302.06476.

Rafael Rafailov, Archit Sharma, Eric Mitchell, Stefano Ermon, Christopher D. Manning, and Chelsea Finn. 2023. Direct preference optimization: Your language model is secretly a reward model.

Leonardo Ranaldi and Andre Freitas. 2024. Aligning large and small language models via chain-of-thought reasoning. In Proceedings of the 18th Conference of the European Chapter of the Association for Computational Linguistics (Volume 1: Long Papers), pages 1812–1827, St. Julian's, Malta. Association for Computational Linguistics.

Baptiste Rozière, Jonas Gehring, Fabian Gloeckle, Sten Sootla, Itai Gat, Xiaoqing Ellen Tan, Yossi Adi, Jingyu Liu, Tal Remez, Jérémy Rapin, Artyom Kozhevnikov, Ivan Evtimov, Joanna Bitton, Manish Bhatt, Cristian Canton Ferrer, Aaron Grattafiori, Wenhan Xiong, Alexandre Défossez, Jade Copet, Faisal Azhar, Hugo Touvron, Louis Martin, Nicolas Usunier, Thomas Scialom, and Gabriel Synnaeve. 2023. Code llama: Open foundation models for code.

Freda Shi, Mirac Suzgun, Markus Freitag, Xuezhi Wang, Suraj Srivats, Soroush Vosoughi, Hyung Won Chung, Yi Tay, Sebastian Ruder, Denny Zhou, Dipanjan Das, and Jason Wei. 2023. Language models are multilingual chain-of-thought reasoners. In The Eleventh International Conference on Learning Representations.

Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. 2024. Reflexion: Language agents with verbal reinforcement learning. Advances in Neural Information Processing Systems, 36.

Shaden Smith, Mostofa Patwary, Brandon Norick, Patrick LeGresley, Samyam Rajbhandari, Jared Casper, Zhun Liu, Shrimai Prabhumoye, George Zerveas, Vijay Korthikanti, et al. 2022. Using deepspeed and megatron to train megatron-turing nlg 530b, a large-scale generative language model. arXiv preprint arXiv:2201.11990.

Kaya Stechly, Matthew Marquez, and Subbarao Kambhampati. 2023. Gpt-4 doesn't know it's wrong: An analysis of iterative prompting for reasoning problems.

Yusheng Su, Chi-Min Chan, Jiali Cheng, Yujia Qin, Yankai Lin, Shengding Hu, Zonghan Yang, Ning Ding, Xingzhi Sun, Guotong Xie, Zhiyuan Liu, and Maosong Sun. 2023. Exploring the impact of model scaling on parameter-efficient tuning.

Xianghui Sun, Yunjie Ji, Baochang Ma, and Xiangang Li. 2023. A comparative study between full-parameter and lora-based fine-tuning on chinese instruction data for instruction following large language model.

Alon Talmor, Jonathan Herzig, Nicholas Lourie, and Jonathan Berant. 2019. CommonsenseQA: A question answering challenge targeting commonsense knowledge. In Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long and Short Papers), pages 4149–4158, Minneapolis, Minnesota. Association for Computational Linguistics.

Hugo Touvron, Thibaut Lavril, Gautier Izacard, Xavier Martinet, Marie-Anne Lachaux, Timothée Lacroix, Baptiste Rozière, Naman Goyal, Eric Hambro, Faisal Azhar, Aurelien Rodriguez, Armand Joulin, Edouard Grave, and Guillaume

Lample. 2023a. Llama: Open and efficient foundation language models.

Hugo Touvron, Louis Martin, Kevin Stone, Peter Albert, Amjad Almahairi, Yasmine Babaei, Nikolay Bashlykov, Soumya Batra, Prajjwal Bhargava, Shruti Bhosale, et al. 2023b. Llama 2: Open foundation and fine-tuned chat models. arXiv preprint arXiv:2307.09288.

Karthik Valmeekam, Matthew Marquez, and Subbarao Kambhampati. 2023. Can large language models really improve by self-critiquing their own plans?

Danqing Wang and Lei Li. 2023. Learning from mistakes via cooperative study assistant for large language models. In Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing, pages 10667–10685.

Ke Wang, Houxing Ren, Aojun Zhou, Zimu Lu, Sichun Luo, Weikang Shi, Renrui Zhang, Linqi Song, Mingjie Zhan, and Hongsheng Li. 2023a. Mathcoder: Seamless code integration in llms for enhanced mathematical reasoning.

Ruida Wang, Wangchunshu Zhou, and Mrinmaya Sachan. 2023b. Let's synthesize step by step: Iterative dataset synthesis with large language models by extrapolating errors from small models.

Xinyi Wang, Lucas Caccia, Oleksiy Ostapenko, Xingdi Yuan, and Alessandro Sordoni. 2023c. Guiding language model reasoning with planning tokens. arXiv preprint arXiv:2310.05707.

Xuezhi Wang, Jason Wei, Dale Schuurmans, Quoc V Le, Ed H Chi, Sharan Narang, Aakanksha Chowdhery, and Denny Zhou. 2022. Self-consistency improves chain of thought reasoning in language models. In The Eleventh International Conference on Learning Representations.

Zengzhi Wang, Rui Xia, and Pengfei Liu. 2023d. Generative ai for math: Part i – mathpile: A billion-token-scale pretraining corpus for math.

Zhaoyang Wang, Shaohan Huang, Yuxuan Liu, Jiahai Wang, Minghui Song, Zihan Zhang, Haizhen Huang, Furu Wei, Weiwei Deng, Feng Sun, and Qi Zhang. 2023e. Democratizing reasoning ability: Tailored learning from large language model.

Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Fei Xia, Ed Chi, Quoc V Le, Denny Zhou, et al. 2022. Chain-of-thought prompting elicits reasoning in large language models. Advances in Neural Information Processing Systems, 35:24824–24837.

Can Xu, Qingfeng Sun, Kai Zheng, Xiubo Geng, Pu Zhao, Jiazhan Feng, Chongyang Tao, and Daxin Jiang. 2023. Wizardlm: Empowering large language models to follow complex instructions.

Aiyuan Yang, Bin Xiao, Bingning Wang, Borong Zhang, Chao Yin, Chenxu Lv, Da Pan, Dian Wang, Dong Yan, Fan Yang, et al. 2023. Baichuan 2: Open large-scale language models. arXiv preprint arXiv:2309.10305.

Longhui Yu, Weisen Jiang, Han Shi, Jincheng Yu, Zhengying Liu, Yu Zhang, James T Kwok, Zhenguo Li, Adrian Weller, and Weiyang Liu. 2023. Metamath: Bootstrap your own mathematical questions for large language models. arXiv preprint arXiv:2309.12284.

Lifan Yuan, Ganqu Cui, Hanbin Wang, Ning Ding, Xingyao Wang, Jia Deng, Boji Shan, Huimin Chen, Ruobing Xie, Yankai Lin, Zhenghao Liu, Bowen Zhou, Hao Peng, Zhiyuan Liu, and Maosong Sun. 2024. Advancing llm reasoning generalists with preference trees.

Zheng Yuan, Hongyi Yuan, Chengpeng Li, Guanting Dong, Chuanqi Tan, and Chang Zhou. 2023. Scaling relationship on learning mathematical reasoning with large language models. arXiv preprint arXiv:2308.01825.

Xiang Yue, Xingwei Qu, Ge Zhang, Yao Fu, Wenhao Huang, Huan Sun, Yu Su, and Wenhu Chen. 2023. Mammoth: Building math generalist models through hybrid instruction tuning.

Susan Zhang, Stephen Roller, Naman Goyal, Mikel Artetxe, Moya Chen, Shuohui Chen, Christopher Dewan, Mona Diab, Xian Li, Xi Victoria Lin, et al. 2022. Opt: Open pre-trained transformer language models. arXiv preprint arXiv:2205.01068.

Denny Zhou, Nathanael Schärli, Le Hou, Jason Wei, Nathan Scales, Xuezhi Wang, Dale Schuurmans, Claire Cui, Olivier Bousquet, Quoc V

Le, et al. 2022. Least-to-most prompting enables complex reasoning in large language models. In The Eleventh International Conference on Learning Representations.

This is the Appendix of the paper: *Can LLMs Learn From Mistakes? An Empirical Study on Reasoning Tasks*.

A Discussion

Here, we discuss further about the insights from our exploration on learning from mistakes.

A.1 LLMs for Self-Correction

Recently, much work has investigated the behavior of advanced LLMs (e.g., GPT-4) on correcting mistakes generated by themselves (Valmeekam et al., 2023; Stechly et al., 2023; Huang et al., 2023). We also conduct further analysis on self-correction performance based on our correction data (detailed in Appendix D.8). These work and our analysis drew the same conclusion: the most powerful LLMs by now still struggle to perform self-correction. To achieve more reliable utilization of self-correction, we think that there are mainly three directions. (1) Inject external supervision to verify the correcting process, such as using the labeled final answers (which is applied in our work) or incorporating human feedback. (2) Train a process-based verifier to judge the quality of self-correction process. Lightman et al. (2023) has demonstrated the great potential of verifier-based method. (3) Develop trust-worth LLMs that can at least honestly tell us what it can solve and what does not.

A.2 Training with Feedback

To align the behavior of LLMs with human expectations, existing work has tried to collect feedback for the model-generated contents and inject these feedback back into the model through various techniques, such as PPO (Lu et al., 2022), RLHF (OpenAI, 2023b) and DPO (Rafailov et al., 2023). To reduce human efforts on annotation, some recent work tried to use LLMs to generate feedback, such as RLAIF (Lee et al., 2023). From this view, LEMA can also be regarded as injecting the feedback from more powerful LLMs (i.e., GPT-4) into smaller models (e.g., LLaMA). We highlight one difference here: the injection process of LEMA is just implemented with instruction-based fine-tuning rather than RL-based methods. It sheds light that for large pre-trained models, it can directly and effectively learn from the comparison between unexpected and expected contents through the input-output fine-tuning process. This can much save the researchers effort to specially design the learning algorithms.

Table 3: Statistics of data sizes for the five tasks in our experiments (without question evolution).

Task	CoT Data	Correction Data	Test Data
GSM8K (Cobbe et al., 2021)	32,421	12,523	1,319
MATH (Hendrycks et al., 2021)	20,009	6,306	5,000
SVAMP (Patel et al., 2021)	-	-	1,000
ASDiv (Miao et al., 2020)	-	-	2,084
CSQA (Talmor et al., 2019)	10,536	7,241	1,221

A.3 Learning From the World Model

Recent advancements in LLMs have enabled them to perform a step-by-step approach in problem-solving. However, this multi-step generation process does not inherently imply that LLMs possess strong reasoning capabilities, as they may merely emulate the superficial behavior of human reasoning without genuinely comprehending the underlying logic and rules necessary for precise reasoning. This incomprehension results in mistakes during the reasoning process and necessitates the assistance of a "world model" that possesses a consciousness prior about the logic and rules governing the real world. From this perspective, our LEMA framework employs GPT-4 as a "world model" to teach smaller models in adhering to these logic and rules, rather than merely mimicking the step-by-step behavior.

B Additional Examples

B.1 Examples in Human Evaluation

Figure 14 illustrates the quality levels of three example corrections.

B.2 Input-Output Formats for Fine-Tuning

Figure 8 illustrate the input-output formats of CoT data and correction data, respectively. Note that during the fine-tuning process, the input part serves as a prompt and only the loss in the output part participates in the back-propagation.

B.3 Evolution Prompt

Figure 9 illustrates our prompt used for evolving new questions from the given seed question.

C More Details For Experimental Setup

C.1 Data Statistics

Table 3 illustrates basic statics about the tasks and data (without question evolution).

Input-Output Format of CoT Data

Input:

Please solve the following math problem.

Question: Natalia sold clips to 48 of her friends in April, and then she sold half as many clips in May. How many clips did Natalia sell altogether in April and May?

Answer: Let's think step by step.

Output:

Natalia sold 48 / 2 = 24 clips in May. Natalia sold 48 + 24 = 72 clips altogether in April and May. The answer is 72.

Input-Output Format of Correction Data

Input:

For the following math problem, the original solution is incorrect. Please identify the incorrect step, explain why it is incorrect, and correct the original solution starting from the incorrect step.

Question: Courtney liked to collect marbles. She kept them in mason jars. One jar had 80 marbles. Her second jar had twice that amount. She just started her third jar which currently has 1/4 the amount of her first jar. How many marbles does she have in total?

Original Solution:

{

Step 1: 80 marbles in the first jar, 2 x 80 = 160 marbles in the second jar.

Step 2: The third jar has 1/4 of the first jar, which is 80 / 4 = 20 marbles.

Step 3: So, Courtney has 160 + 20 = 180 marbles in total.

Step 4: The answer is 180.

}

Incorrect Step:

Output:

Step 3.

Explanation: The calculation in Step 3 is incorrect because it only adds the marbles from the second and third jars. The marbles from the first jar should also be included in the total.

Correct Solution:

{

…

Step 3: So, Courtney has 80 (first jar) + 160 (second jar) + 20 (third jar) = 260 marbles in total.

Step 4: The answer is 260.

}

Note: The ellipsis here are tokens that were actually used in the fine-tuning process, as we suppose that simply copying pre-steps is not much informative for learning.

Figure 8: The input-output formats for our CoT data and correction data, respectively. The input part serves as a prompt and only the loss in the output part participates in the back-propagation.

C.2 Evaluation on ASDiv

As mentioned in our setup, the original version of ASDiv contains 2,305 questions and part of them lead to non-numerical answers. For instance, for the question "Mrs. Hilt has two pennies, two dimes, and two nickels. Jacob has four pennies, one nickel, and one dime. Who has more money?", the answer is the string value "Mrs. Hilt"; for the question "Tessa has 4 apples. Anita gave her 5 more. She needs 10 apples to make a pie. Does she have enough to make a pie?", the answer is a Boolean value "False". As our models are trained on data derived from GSM8K where questions are all leading to numerical answers, it is reasonable that these models can not generate non-numerical answers. Therefore, for evaluation on ASDiv, we filter out questions with non-numerical answers and finally leave 2,084 questions. Specifically, for the question-answer pair in ASDiv, it will be filtered out if the answer can not be successfully recognized by the Python function $\mathrm{float}(\cdot)$.

C.3 Data Construction For CSQA

The original training examples in CSQA only contain the labeled final answers without rationales. Therefore, we need to generate CoT for the training examples. We first annotate rationales for four training examples. Figure 10 shows one annotated example. Specifically, the CoT contain three parts: the explanation to each candidate answers, the predicted final answer, and the reason to choose this answer. Then, we utilize GPT-4 to generate rationales for other training examples and filter out rationales that do not contain the correct final answers. For generating correction data, we do not require GPT-4 to explicitly identify the position of mistake. It is because the CoT for commonsense questions does not exhibit a clear step-wise manner,

Prompt For Question Evolution

I want you to act as a math teacher. You need to create relevant math questions for the given question. The created math question must be reasonable, without unknown variables, and must be understood to answer in pure number by humans. Ensure that the created questions are **diverse** from the original one. Ensure that the created questions use **different natural language expressions**.

#Given Question#: q_i

#Created Question#:

Figure 9: The prompt for evolving questions.

Annotated CoT Example For CSQA

Please answer the following commonsense question. Please first explain each candidate answer, then select only one answer that is most relevant to the question and provide reasons.

Question: The fox walked from the city into the forest, what was it looking for?

Candidate Answers: pretty flowers, hen house, natural habitat, storybook, dense forest.

Explanations:

Pretty flowers - This answer suggests that the fox might be looking for beautiful flowers. However, it is not common for a fox to seek out flowers, as they are carnivorous animals and do not have a particular interest in plants.

Hen house - This answer implies that the fox might be searching for a hen house to find food, such as chickens. Foxes are known to prey on small animals, and a hen house would be a potential source of food for them.

Natural habitat - This answer suggests that the fox is looking for its natural habitat, which is typically a forest or wooded area. Foxes are wild animals and are more comfortable in their natural environment than in a city.

Storybook - This answer implies that the fox is searching for a storybook, which is an inanimate object and not something a fox would be interested in or capable of finding.

Dense forest - This answer is similar to the natural habitat option, suggesting that the fox is looking for a dense forest, which could be a part of its natural habitat.

Reasons and Final Answer:

The most relevant answer to the question is that the fox was looking for its natural habitat. This is because foxes are wild animals and are more comfortable in their natural environment, such as a forest, than in a city. The other candidate answers either do not align with the natural behavior of a fox (pretty flowers, storybook) or are too specific (hen house, dense forest) without enough context to support them as the most likely answer.

The answer is natural habitat.

Figure 10: One annotated CoT example for CSQA.

and our ablation study on math tasks have showed that this information is less influential to the final performance.

C.4 Experimental Settings for Evolution

In Figure 6, besides the correction-centric evolution introduced in Section 2.2, we also compare with the general evolution strategy applied in previous work (Xu et al., 2023; Yu et al., 2023; Li et al., 2023a). For a fair comparison, the number of seed questions is kept the same for both evolution strategies (i.e., 10K). We also tried the Llemma (Azerbayev et al., 2023) model which has been pre-trained on a math-related corpus (such as arXiv papers). We fully fine-tune LLMs as the correction data scale has been much increased.

C.5 Another Round of Correction-Centric Evolution

To explore the scaling trend of LEMA, we take another round of correction-centric evolution to expand correction data. The second round takes the same 10K seed questions as the first round. The only difference is that we replace the vanilla model as the fine-tuned models from the first round to collect inaccurate reasoning paths.

D More Results and Analysis

D.1 Performances of Best Three Checkpoints

Table 4 shows the performances of the best three checkpoints saved during the fine-tuning process along with the average of three results. It demonstrates that our main results are not caused by soem random disturbances during training.

D.2 Training Curves

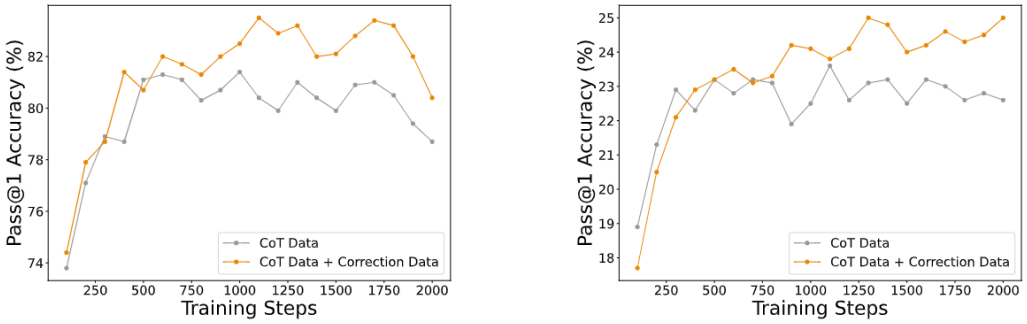
Figure 11 shows the performance curves of LLaMA-2-70B during 2,000 fine-tuning steps. It shows that adding correction data leads to clear improvements during training. These consistent improvements demonstrate that the effectiveness of our correction data is robust to the random disturbances during training.

D.3 Performances on OOD Tasks

Despite the three tasks in our main text, here we take experiments on another two math tasks SVAMP (Patel et al., 2021) and ASDiv (Miao et al., 2020). We use the model fine-tuned for GSM8K

Table 4: Performances of the best three checkpoints during the fine-tuning process and the average of three results.

Model	Method (with QLoRA)	GSM8K		MATH	
		1st / 2nd / 3rd	Avg.	1st / 2nd / 3rd	Avg.
LLaMA-2-70B (Touvron et al., 2023b)	Straightforward Learning	81.4 / 81.3 / 81.1	81.3	23.6 / 23.2 / 23.2	23.2
	+ Learning From Mistakes	83.5 / 83.4 / 83.2	83.4 (+2.1)	25.0 / 25.0 / 24.6	24.9 (+1.7)
LLaMA-65B (Touvron et al., 2023a)	Straightforward Learning	76.2 / 76.2 / 75.7	76.0	19.7 / 19.7 / 19.2	19.5
	+ Learning From Mistakes	77.9 / 77.3 / 77.2	77.5 (+1.5)	20.8 / 20.3 / 20.2	20.4 (+0.9)
CodeLLaMA-34B (Rozière et al., 2023)	Straightforward Learning	68.8 / 68.5 / 68.2	68.5	19.1 / 19.0 / 18.9	19.0
	+ Learning From Mistakes	71.7 / 71.0 / 70.9	71.2 (+2.7)	20.4 / 20.2 / 20.0	20.2 (+1.2)
LLaMA-2-13B (Touvron et al., 2023b)	Straightforward Learning	62.9 / 62.7 / 62.7	62.8	12.2 / 11.9 / 11.8	12.0
	+ Learning From Mistakes	65.7 / 65.2 / 65.0	65.3 (+2.5)	12.6 / 12.6 / 12.4	12.5 (+0.5)
LLaMA-2-7B (Touvron et al., 2023b)	Straightforward Learning	52.6 / 52.5 / 52.5	52.5	8.7 / 8.5 / 8.5	8.6
	+ Learning From Mistakes	54.1 / 53.7 / 53.6	53.8 (+1.3)	9.4 / 8.9 / 8.8	9.0 (+0.4)



(a) GSM8K (b) MATH

Figure 11: The performance curves of LLaMA-2-70B during 2,000 fine-tuning steps.

and take these two tasks for out-of-distribution (OOD) evaluations.

SVAMP (Patel et al., 2021) consists of questions with short NL narratives as state descriptions. For evaluation on SVAMP, we use the same training data as for GSM8K and take all 1,000 examples in SVAMP as test cases.

ASDiv (Miao et al., 2020) is a diverse math dataset in terms of both language patterns and problem types for evaluating. For evaluation on ASDiv, we use the same training data as for GSM8K and test on 2,084 examples in ASDiv[7].

Table 5 shows the results on these two OOD tasks. These improvements indicate that LEMA has a certain extent of generalizability in the out-of-distribution scenarios.

D.4 SFT v.s. DPO

Our experiments are mainly conducted under the SFT paradigm. We do not take DPO because it can not utilize the explanations and reasons of mistakes in our correction data. Here, we provide some preliminary explorations showing that DPO does not suit our setting.

Specifically, we conduct the standard DPO training and also try a variant of DPO with a modification on the loss function, defined in Equation 4,

$$\mathcal{L}(\pi_\theta; \pi_{ref}) = \quad (4)$$
$$-\mathbb{E}_{(x,y_w,y_l)\sim D}[\log \sigma(\beta \log \frac{\pi_\theta(y_w|x)}{\pi_{ref}(y_w|x)}$$
$$-\beta \log \frac{\pi_{ref}(y_l|x)}{\pi_\theta(y_l|x)})] \; \boldsymbol{-\gamma \log P_\theta(y_w|x)} \, .$$

Such a modification is inspired by the recent work (Pal et al., 2024; Yuan et al., 2024). It seems that applying DPO on quite similar positive and negative examples requires an additional penalty term to avoid the reduction of the model's likelihood of the preferred examples.

Table 6 shows the performances of DPO training. It shows that applying DPO to perform learning from mistakes can outperforms straightforward learning, but cannot outperform SFT training which could further utilize the explanations and reasons of mistakes during training.

[7]The original ASDiv contains 2,305 examples and we filter out non-numerical examples, detailed in Appendix C.2.

Table 5: Results on two out-of-distribution tasks.

Model	Method (with QLoRA)	SVAMP	ASDiv
LLaMA-2-70B (Touvron et al., 2023b)	Straightforward Learning	80.3	80.7
	+ Learning From Mistakes	81.6 (+1.3)	82.2 (+1.5)
LLaMA-65B (Touvron et al., 2023a)	Straightforward Learning	71.9	77.4
	+ Learning From Mistakes	72.8 (+0.9)	77.7 (+0.3)
CodeLLaMA-34B (Rozière et al., 2023)	Straightforward Learning	67.4	73.9
	+ Learning From Mistakes	72.0 (+4.6)	74.4 (+0.5)
LLaMA-2-13B (Touvron et al., 2023b)	Straightforward Learning	58.0	67.8
	+ Learning From Mistakes	62.0 (+4.0)	71.1 (+3.3)
LLaMA-2-7B (Touvron et al., 2023b)	Straightforward Learning	53.0	63.8
	+ Learning From Mistakes	54.1 (+1.1)	65.5 (+1.7)

Table 6: Results with DPO training on GSM8K.

Model	Method (with QLoRA)	GSM8K
Mistral-7B	Straightforward Learning (SFT)	68.2
	+ Learning From Mistakes (SFT)	71.3 (+3.1)
	+ Learning From Mistakes (standard DPO)	69.1 (+0.9)
	+ Learning From Mistakes (modified DPO)	70.2 (+2.0)

Table 7: Performances with the same size of training tokens (5.8M) on GSM8K (with QLoRA for training).

Model	Data	Acc (%)
LLaMA-2-70B	CoT-5.8M	82.1
	LEMA-5.8M	**83.5 (+1.4)**
LLaMA-2-13B	CoT-5.8M	64.2
	LEMA-5.8M	**65.7 (+1.5)**

D.5 Training-Token Efficiency

Despite controlling the training data sizes to be the same, we also investigate the training-token efficiency of learning from mistakes compared with only applying straightforward learning. Notice that the target-side length of mistake-correction data is generally longer than CoT data, so incorporating learning from mistakes will have slightly more training tokens than straightforward learning under the same data size. Specifically, CoT-45K has 5.4M training tokens and LEMA-45K has 5.8M (a ∼7% relative increment). To conduct the comparison under the same size of training tokens, we construct CoT-5.8M by sampling more reasoning paths (following Section 2.3) to add into CoT-45K. Table 7 shows that LEMA still outperforms CoT-alone fine-tuning with the same number of training tokens. Note that this comparison is under an unfavorable setup for LEMA as it increases the training samples for CoT-alone fine-tuning. The improvements in Table 7 further support the non-homogeneous effectiveness of CoT data and mistake-correction data. Moreover, we notice that augmenting more reasoning paths for LLaMA-2-70B does not continuously boost the model performance on GSM8K. To validate this, we further expand CoT-5.8M to CoT-6.8M and have a 82.2% accuracy. Such an observation is in line with the Yu et al. (2023). We suppose that this is because sampling too many reasoning paths for the same question will only bring redundant information to the training.

D.6 Performances with Specialized Models

Table 8 contains more results with specialized models. Another interesting finding in Table 8 is that the performance of LLaMA-2-70B + LEMA can be comparable with MuggleMath-70B (Li et al., 2023a) and MetaMath-70B (Yu et al., 2023). Note that these two specialized LLMs also take the LLaMA-2-70B as the backbone model while their training data sizes are much larger than LEMA: MuggleMath has ∼220K CoT data and MetaMath has ∼400K CoT data, while LEMA only has ∼70K CoT + correction data for math problems. This comparison further supports the non-homogeneous effectiveness between CoT data and correction data.

D.7 Additional Analysis to LEMA

LEMA can still bring improvements to Straightforward Learning if the distributions of questions are controlled the same. In our default setting, correction data contains more challenging questions that can not be easily solved by various LLMs. This leads to a distribution shift on the difficulty of questions in training data. As Wang et al. (2023b) indicated that this distribution shift can also benefit fine-tuning LLMs, we also mitigate the influence from question distribution shift to further clarify the effectiveness of LEMA. Our

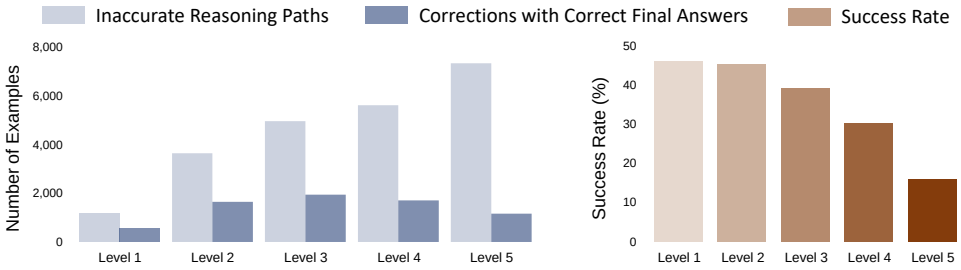


Figure 12: Statistics of generated correction data according to different difficulty levels in MATH. **Left:** The number of collected inaccurate reasoning paths and generated corrections with correct final answers under different difficulty levels. **Right:** The success rate for correcting inaccurate reasoning paths under different difficulty levels.

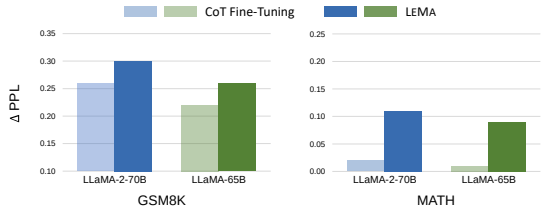


Figure 13: The differences between the PPLs (ΔPPL) on mistaken CoT and correct CoT. A higher difference indicate that the model can better avoid the mistakes.

ablation setting CoT-45K can be used to clarify this point: its additional CoT data are just converted from correction data, thus the question distributions of CoT-45K and our default LEMA-45K are exactly the same. Therefore, the results in Figure 5 under 45K data size demonstrate that LEMA still outperforms CoT-alone fine-tuning when the influence from question distribution shift is kept the same.

QLoRA fine-tuning cannot fully "digest" a large amount of correction data. As shown in Figure 7, as the correction data expands, the gap between full-fine-tuning and QLoRA fine-tuning increases. Such an observation is not well aligned with the conclusions of some existing work. Some work indicated that if the model size is large enough, parameter-efficient fine-tuning (PEFT) can achieve comparable performance with fine-tuning (Lester et al., 2021; An et al., 2022; Sun et al., 2023; Su et al., 2023; Artur Niederfahrenhorst and Ahmad, 2023). We suppose the property of correction data causes the inconsistency in observations. Specifically, correction data is just auxiliary data that do not directly contribute to the in-task training. We suppose that models with PEFT can "eat" a large amount of correction data but cannot fully "digest" them. As a results, the training on correction data with PEFT might not effectively contribute to the forward reasoning process.

The comparison learned in the correction data also influences the CoT generation. During training on the correction data, LLMs could be aware of the comparison between the correct and incorrect CoT. We suppose such kind of comparison can take effect during CoT generation. Based on this intuition, we evaluate the differences between PPLs defined as follows,

$$\Delta\mathrm{PPL}(\mathcal{C};\theta) = \frac{1}{||\mathcal{C}||}\sum_{(q_i,\widetilde{r}_i,c_i)\in\mathcal{C}}[\mathrm{PPL}(\widetilde{r}_i|q_i;\theta) - \mathrm{PPL}(r_i|q_i;\theta)], \quad (5)$$

where $\mathcal{C}$ is a set of correction data, θ represents the model parameters after fine-tuning, $\mathrm{PPL}(y|x;\theta)$ returns the perplexity on y with x as the context, $\widetilde{r}_i$ is one mistaken CoT for the question q_i, and r_i is the correct CoT extracted from the correction c_i. We calculate ΔPPL for fine-tuned LLaMA-2-70B and LLaMA-65B, based on the correction data for GSM8K and MATH. Figure 13 shows ΔPPL for different fine-tuned models. It shows that LEMA consistently leads to a higher ΔPPL than CoT-alone fine-tuning.

D.8 Further Analysis on Corrector

In our default setting, we take GPT-4 as the corrector model and our human evaluation in Section 2.1 supports this choice. In the following, we provide further analysis on the choice and behavior of the corrector model. Specifically, we want to answer the following research questions: **RQ1:** Can we use a less powerful model as the corrector model? **RQ2:** How well does GPT-4 perform in self-correction? **RQ3:** How well does GPT-4 correct inaccurate reasoning paths for challenging questions?

Table 8: Math reasoning performances of various LLMs.

Model	GSM8K	MATH
closed-source models		
GPT-4 (OpenAI, 2023b)	92.0	42.5
Claude-2 (Anthropic, 2023)	88.0	-
Flan-PaLM-2 (Anil et al., 2023)	84.7	33.2
GPT-3.5-Turbo (OpenAI, 2023a)	80.8	34.1
PaLM-2 (Anil et al., 2023)	80.7	34.3
open-source models		
LLaMA-2-7B (Touvron et al., 2023b)	14.6	2.5
Baichuan-2-7B (Yang et al., 2023)	24.5	5.6
SQ-VAE-7B (Wang et al., 2023c)	40.0	7.0
RFT-7B (Yuan et al., 2023)	50.3	-
Qwen-7B (Alibaba, 2023)	51.6	-
LLaMA-2-7B + LEMA (ours)	54.1	9.4
WizardMath-7B (Luo et al., 2023)	54.9	10.7
WizardMath-7B + LEMA (ours)	55.9	11.9
LLaMA-2-13B (Touvron et al., 2023b)	28.7	3.9
SQ-VAE-13B (Wang et al., 2023c)	50.6	8.5
Baichuan-2-13B (Yang et al., 2023)	52.8	10.1
RFT-13B (Yuan et al., 2023)	54.8	-
WizardMath-13B (Luo et al., 2023)	63.9	14.0
LLaMA-2-13B + LEMA (ours)	65.7	12.6
MetaMath-13B (Yu et al., 2023)	72.3	22.4
MetaMath-13B + LEMA (ours)	73.2	22.7
LLaMA-2-70B (Touvron et al., 2023b)	56.8	13.5
RFT-70B (Yuan et al., 2023)	64.8	-
WizardMath-70B (Luo et al., 2023)	81.6	22.7
MuggleMath-70B (Li et al., 2023a)	82.3	-
MetaMath-70B (Yu et al., 2023)	82.3	26.6
LLaMA-2-70B + LEMA (ours)	83.5	25.0
WizardMath-70B + LEMA (ours)	84.2	**27.1**
MetaMath-70B + LEMA (ours)	**85.4**	26.9

Less powerful models are not suitable for generating corrections. Despite GPT-4, we have also tried leveraging GPT-3.5-Turbo as the corrector model and assess the quality of generated corrections. We take another round of human evaluation on 20 corrections generated by GPT-3.5-Turbo and find that nearly half are of poor quality. Therefore, we just call GPT-4 for correction generation although it is much more expensive than GPT-3.5-Turbo. We believe it is a valuable research direction to explore how to generate high-quality corrections without GPT-4.

GPT-4 can correct its own mistakes but with a low success rate. Specifically, for 2,696 inaccurate reasoning paths generated by GPT-4 on MATH training set, we finally get 217 corrections with correct final answers. It means that GPT-4 only achieves 8.0% success rate for self-correction. Compared with this low success rate for self-correction, GPT-4 can more effectively correct mistakes from less powerful models, such as LLaMA-2-70B (37.5% success rate on MATH) and GPT-3.5-Turbo (26.9% success rate on MATH). One possible reason for the low success rate of self-correction is that the mistakes generated by GPT-4 are from more challenging questions, thus these mistakes are naturally harder for correcting.

GPT-4 still struggles to correct inaccurate reasoning paths for challenging questions. The math problems in MATH can be categorized into five levels of difficulty: Level 1 for the easiest problems and Level 5 for the most challenging ones. Figure 12 shows statistics of our correction data on MATH according to different difficulty levels. As the difficulty increased from Level 1 to Level 5, the number of collected inaccurate reasoning paths increased, while the number of correct corrections (i.e., corrections for which the final answer is correct) first increases and then decreases. We also calculate the success rate for correcting mistakes under each difficulty level, dividing the number of correct corrections by the total number of collected reasoning paths. Figure 12 shows that the success rate significantly drops with increasing the difficulty. These statistics reveals that there is still huge room for improving contemporary LLMs on correcting mistakes.

Generated Correction (Excellent)

Question: Courtney liked to collect marbles. She kept them in mason jars. One jar had 80 marbles. Her second jar had twice that amount. She just started her third jar which currently has 1/4 the amount of her first jar. How many marbles does she have in total?

Original Solution:

Step 1: 80 marbles in the first jar, 2 x 80 = 160 marbles in the second jar.

Step 2: The third jar has 1/4 of the first jar, which is 80 / 4 = 20 marbles.

Step 3: So, Courtney has 160 + 20 = 180 marbles in total.

Step 4: The answer is 180.

Incorrect Step: Step 3.

Explanation: The calculation in Step 3 is incorrect because it only adds the marbles from the second and third jars. The marbles from the first jar should also be included in the total.

Correct Solution:

…

Step 3: So, Courtney has 80 (first jar) + 160 (second jar) + 20 (third jar) = 260 marbles in total.

Step 4: The answer is 260.

Generated Correction (Good)

Question: Johnny is a dog walker. He can walk 3 dogs at once. He gets paid $15 for a 30-minute walk and $20 for a 60-minute walk. Johnny works for 4 hours per day. If he always walks the maximum number of dogs possible and 6 dogs have 60-minute walks per day, how much money does he make in a week where he works 5 days?

Original Solution:

Step 1: 4 hours per day * 5 days = 20 hours per week.

Step 2: Johnny can walk 3 dogs at once, so he can walk 20 / 3 = 6 three-dog walks per day.

…

Step 6: The answer is $360.

Incorrect Step: Step 2.

Explanation: The incorrect step is Step 2. The calculation used in this step is not correct for finding the number of three-dog walks per day. Instead, we should calculate the number of 30-minute and 60-minute walks Johnny does.

Correct Solution:

…

Step 2: Johnny works for 4 hours per day, which is 240 minutes. He walks 6 dogs for 60 minutes each, which takes 120 minutes. This leaves 120 minutes for 30-minute walks.

…

Step 6: The answer is 1500.

Note: The explanation is reasonable and the final answer is correct. But the new solution has issues on continuity: the new Step 2 does not exactly follow the original Step 1.

Generated Correction (Poor)

Question: Tom, Tim, and Paul are collecting photos of cars. Paul has 10 photos more than Tim. Tim has one hundred photos less than the total amount of photos which is 152. How many photos does Tom have?

Original Solution:

Step 1: 152 - 100 = 52.

Step 2: Tom has 52 photos.

Step 3: The answer is 52.

Incorrect Step: Step 1.

Explanation: The step is incorrect because it subtracts 100 from the total number of photos (152) to find the number of photos Tim has. However, the problem states that Tim has 100 photos less than the total amount of photos, not the total amount of photos minus 100.

Correct Solution:

…

Step 5: The answer is 38.

Note: Step 1 is correct and the mistake step is Step 2, because Tim, not Tom, has 52 photos.

Figure 14: Some examples of generated corrections and their quality levels under our human evaluation.